

Chapter 02

Introduction to C++20

Programming

Learning Objectives

- Write simple C++ applications.
- Use input and output statements.
- Use fundamental data types.
- Use arithmetic operators.
- Understand the precedence of arithmetic operators.
- Write decision-making statements.
- Use relational and equality operators.
- Begin appreciating the “Objects Natural” learning approach by creating and using objects of the C++ standard library's `class`

Introduction

- This chapter presents several code examples that demonstrate how your programs can **display messages** and **obtain data** from the user for processing.
 - The first three examples output messages on the screen.
 - The fourth example obtains two numbers from a user at the keyboard, calculates their sum and displays the result—the accompanying discussion introduces C++’s arithmetic operators.
 - The fifth example demonstrates decision-making by showing how to compare two numbers, then display messages based on the comparison results.

Introduction

- The “Objects Natural” Learning Approach
 - In your programs, you’ll create and use many objects of preexisting carefully-developed-and-tested classes that enable you to perform significant tasks with minimal code.
 - These classes typically come from:
 - ◆ the C++ standard library,
 - ◆ platform-specific libraries, and
 - ◆ free third-party libraries created by the massive open-source communities.
 - To help you appreciate this programming style, you’ll conveniently create and use objects of preexisting C++ standard library classes. We call this the “Objects Natural” approach.
 - ◆ You’ll begin by creating and using objects in this chapter’s final example.

First Program in C++: Displaying a Line of Text

- Comment

- Document your programs- help other people read and understand.
- Ignore by the compiler- do not cause the computer to perform any action when the program running.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- Comment

- Single-line comment: `//`
- Multiline comments: `/* ... */`

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- Preprocessor Directive

- `#include <iostream>` notifies the preprocessor to include in the program the contents of the input/output stream header, containing required information when compiling program with I/O.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- Blank Lines and Whitespace

- `#include <iostream>` notifies the preprocessor to include in the program the contents of the input/output stream header, containing required information when compiling program with I/O.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- The main function

- Part of every C++ program, the starting point
- Parentheses after main indicate that it's a **function**
- int means it returns an integer value
- Begins and ends with **{ }**, between are instructions

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- An Output Statement

- Line 6 displays the characters contained between the double quotation marks, called a **string**, a **character string** or a **string literal**.
- The entire line is a **statement**.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- An Output Statement

- Syntax error

- ◆ Omitting the semicolon is a **syntax error**.
 - ◆ Specifies the rules for creating proper programs
 - ◆ Occurs when the compiler encounters code that violates C++'s language rules (i.e., its syntax).

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- An Output Statement

- Syntax error

- ◆ Syntax errors are also called **compiler errors**, **compile-time errors** or **compilation errors** because the compiler detects them during the compilation phase.
 - ◆ You cannot execute your program until you correct all the syntax errors.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- Indentation

- Indent each function's body one level within the braces that delimit the body.
- Making a program's functional structure stand out, and easier to read.
- Set a convention for the indent size: tab or spaces

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- The `std` Namespace

- The `std::` before `cout` is required when we use names that we've brought into the program from standard-library headers.
- This specifies that we are using the name `cout`, which belongs to `namespace std`.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- The Stream Insertion Operator <<
 - The value to the operator's right (the right operand) is inserted in the output stream.
 - << points toward where the data goes.
 - A string's characters are typically displayed exactly as typed between the quotes.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- Escape Sequences

- When C++ encounters a backslash in a string, it combines the next character with the backslash to form an **escape sequence**.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

First Program in C++: Displaying a Line of Text (Cont.)

- Escape Sequences

Escape sequence	Description
\n	Newline. Positions the screen cursor to the beginning of the next line.
\t	Horizontal tab. Moves the screen cursor right to the next tab stop.
\r	Carriage return. Positions the screen cursor to the beginning of the current line; does not advance to the next line.
\a	Alert. Sounds the system bell.
\\	Backslash. Includes a backslash character in a string.
\'	Single quote. Includes a single-quote character in a string.
\"	Double quote. Includes a double-quote character in a string.

First Program in C++: Displaying a Line of Text.

- The return Statement

- Use to exit a function
- 0 indicates that the program terminated successfully.
- The main function implicitly returns 0 if program execution reaches the closing brace without encountering a return statement -> can be omitted.

```
1 // Text-printing program.
2 #include <iostream> // enables program to output data to the screen
3
4 // function main begins program execution
5 int main() {
6     std::cout << "Welcome to C++!\n"; // display message
7
8     return 0; // indicate that program ended successfully
9 } // end function main
10
```

Welcome to C++!

Another C++ Program: Adding Two Integers

- Our next program obtains two integers typed by a user at the keyboard, computes their sum and outputs the result using `std::cout`.

Another C++ Program: Adding Two Integers (Cont.)

Enter first integer: 45
Enter second integer: 72
Sum is 117

```
1 // Addition program that displays the sum of two integers.
2 #include <iostream> // enables program to perform input and output
3
4 // function main begins program execution
5 int main() {
6     // declaring and initializing variables
7     int number1{0}; // first integer to add (initialized to 0)
8     int number2{0}; // second integer to add (initialized to 0)
9     int sum{0}; // sum of number1 and number2 (initialized to 0)
10
11     std::cout << "Enter first integer: "; // prompt user for data
12     std::cin >> number1; // read first integer from user into number1
13
14     std::cout << "Enter second integer: "; // prompt user for data
15     std::cin >> number2; // read second integer from user into number2
16
17     sum = number1 + number2; // add the numbers; store result in sum
18
19     std::cout << "Sum is " << sum << "\n"; // display sum
20 }
```

Another C++ Program: Adding Two Integers (Cont.)

- Variable Declarations and Braced Initialization

- Lines 7-9 are **declarations**.

```
7   int number1{0}; // first integer to add (initialized to 0)
8   int number2{0}; // second integer to add (initialized to 0)
9   int sum{0}; // sum of number1 and number2 (initialized to 0)
```

- A **variable** is a location in the computer's memory where a value can be stored for use by a program.
- These declarations specify that the variables are data of type **int**, meaning they will hold **integer** (whole number) values, such as 7, -11, 0 and 31914.
- All variables must be declared with a name and a data type.

Another C++ Program: Adding Two Integers (Cont.)

- Variable Declarations and Braced Initialization

- Lines 7–9

```
7   int number1{0}; // first integer to add (initialized to 0)
8   int number2{0}; // second integer to add (initialized to 0)
9   int sum{0}; // sum of number1 and number2 (initialized to 0)
```

initialize each variable to 0 by placing a value in braces ({ and }) immediately following the variable's name. This is known as **braced initialization**, which was introduced in C++11.

- Declarations can be placed **almost anywhere** in a C++ program, but they must appear **before** their corresponding variables are **used** in the program.
 - In legacy C++ programs, you're likely to encounter initialization statements using this **older coding style**:
`int number1 = 0;`
 - Our code examples use braced initialization. In subsequent chapters, we'll discuss various benefits of braced initializers.

Another C++ Program: Adding Two Integers (Cont.)

- Fundamental Types

- We'll soon discuss the type `double` for specifying real numbers and `char` for character data.
 - ◆ Real numbers are numbers with decimal points, such as 3.4, 0.0 and -11.19.
 - ◆ A `char` variable may hold one lowercase letter, uppercase letter, digit or special character (e.g., \$ or *).
 - ◆ Types such as `int`, `double`, `char` and `long` are called fundamental types and are built into C++.
 - ◆ Fundamental-type names typically consist of one or more keywords and must appear in all lowercase letters.
 - ◆ For a complete list of C++ fundamental types and their ranges, see <https://en.cppreference.com/w/cpp/language/types>

Another C++ Program: Adding Two Integers (Cont.)

- Identifiers and Camel-Case Naming
 - A variable name may be any valid identifier.
 - An identifier is a series of characters consisting of letters, digits and underscores (`_`) that does not begin with a digit and is not a keyword.
 - C++ is case-sensitive—uppercase and lowercase letters are different.
 - C++ allows identifiers of any length.
 - Do not begin an identifier with an underscore and a capital letter or two underscores—C++ compilers use such names for their own purposes internally.

Another C++ Program: Adding Two Integers (Cont.)

- Identifiers and Camel-Case Naming

- By convention, variable-name identifiers begin with a lowercase letter, and every word in the name after the first word begins with a capital letter.
 - ◆ For example, `firstNumer` starts its second word, `Number`, with a capital `N`.
 - ◆ This naming convention is known as camel case because the uppercase letters stand out like a camel's humps.
- Choosing meaningful identifiers helps make a program self-documenting—a person can understand the program simply by reading it rather than referring to program comments or external documentation.
- Avoid using abbreviations in identifiers. Although this may require you to type more keystrokes, it improves program readability.

Another C++ Program: Adding Two Integers (Cont.)

- Obtaining the Value from the User

- Output and input in C++ are accomplished with **streams of characters**.

- Line 11

`std::cout << "Enter first integer: ";` // prompt user for data
uses the **standard output stream object—`std::cout`**—
and the **stream insertion operator, `<<`**, to display the
string "Enter first integer: ".

- Thus, when line 11 executes, it sends the stream of characters "Enter first integer: " to `std::cout`, which is normally “connected” to the screen.

- We like to pronounce the preceding statement as “`std::cout` gets the character string "Enter first integer: ".”

Another C++ Program: Adding Two Integers (Cont.)

- Obtaining the Value from the User

- Line 12

- ```
std::cin >> number1; // read first integer from user into number1
```

 uses the **standard input stream object**—  
**`std::cin`**—and the **stream extraction operator**, **`>>`**, to obtain a value from the keyboard.

- Using the stream extraction operator with `std::cin` takes character input from the standard input stream, which is usually the keyboard.
  - We like to pronounce the preceding statement as, “**`std::cin` gives a value to `number1`**” or simply “**`std::cin` gives `number1`**.”

# Another C++ Program: Adding Two Integers (Cont.)

- Obtaining the Value from the User.
  - When the computer executes the statement in line 12, it waits for the user to enter a value for variable `number1`.
  - The user responds by typing an integer (as characters), then pressing the Enter key.
  - The computer converts the character representation of the number to an integer and assigns this value to the variable `number1`.
  - Line 14 displays "Enter second integer: " on the screen, prompting the user to take action.
  - Line 15 obtains a value for variable `number2` from the user.

```
14 std::cout << "Enter second integer: "; // prompt user for data
15 std::cin >> number2; // read second integer from user into number2
```

# Another C++ Program: Adding Two Integers (Cont.)

- Calculate the Sum of the Values Input by the User

- The assignment statement in line 17

```
sum = number1 + number2; // add the numbers; store result in sum
```

calculates the sum of the variables `number1` and `number2` and assigns the result to variable `sum`.

# Another C++ Program: Adding Two Integers (Cont.)

- Displaying the Result

- Line 19

- ```
std::cout << "Sum is " << sum << "\n"; // display sum
```

- displays the character string `Sum is` followed by the numerical value of variable `sum` followed by `std::endl`—a so-called stream manipulator.

- ◆ The name `endl` is an abbreviation for “end line.”

- The `std::endl` stream manipulator outputs a newline, then “**flushes** the output buffer.”

- ◆ This simply means that, on some systems where outputs accumulate in the machine until there are enough to “make it worthwhile” to display on the screen, `std::endl` forces any accumulated outputs to be displayed at that moment.

- ◆ This can be important when the outputs are prompting the user for an action, such as entering data.

Another C++ Program: Adding Two Integers (Cont.)

- Displaying the Result

- Calculations can also be performed in output statements.
- We could have combined the statements in lines 17 and 19 into the statement

`std::cout << "Sum is " << number1 + number2 << std::endl;`
thus eliminating the need for the variable `sum`.

Another C++ Program: Adding Two Integers (Cont.)

- Displaying the Result.

- The stream insertion operator “knows” how to output each type of data.
 - ◆ The statement in line 19 outputs values of different types.
- Using multiple **stream insertion operators** (<<) in a single statement is referred to as **concatenating**, **chaining** or **cascading stream insertion operations**.
- A powerful C++ feature is that users can create their own types called **classes**.
- Users can then “teach” C++ how to input and output values of these new data types using the >> and << operators (this is called **operator overloading**—a topic we explore later).

Another C++ Program: Adding Two Integers (Cont.)

- Arithmetic Operators

Operation	Arithmetic operator	Algebraic expression	C++ expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm or $b \cdot m$	<code>b * m</code>
Division	/	x/y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Remainder	%	$r \bmod s$	<code>r % s</code>

Another C++ Program: Adding Two Integers (Cont.)

- Integer Division

- **Integer division** in which the numerator and the denominator are integers yields an integer quotient.
- $7/4$ evaluates to 1
- $17/5$ evaluates to 3
- Any fractional part in the result is truncated—no rounding occurs.

Another C++ Program: Adding Two Integers (Cont.)

- Remainder Operator

- $x \% y$ yields the remainder after dividing x by y using integer division
- Can be used only with integer operands
- $7 \% 4$ yields 3
- $17 \% 5$ yields 2

Another C++ Program: Adding Two Integers (Cont.)

- Parentheses for Grouping Subexpressions
 - Parentheses group C++ expressions in the same manner as in algebra
 - To multiply a times the quantity $b + c$, write
 - $a * (b + c)$

Another C++ Program: Adding Two Integers (Cont.)

- Rules of Operator Precedence

- C++ applies the operators in arithmetic expressions in a precise order determined by the **rules of operator precedence**—generally the same as in algebra
- Expressions in parentheses evaluate first.
- Multiplication, division and remainder operations evaluate next.
 - ◆ Applied from left-to-right.
- Addition and subtraction operations evaluate last.
 - ◆ Applied from left-to-right.
- **Caution:** If you have an expression such as $(a + b) * (c + d)$ in which two sets of parentheses are not nested, but appear “on the same level,” the C++ Standard does **NOT** specify the order in which these **parenthesized subexpressions** will be evaluated

Another C++ Program: Adding Two Integers.

- Operator Grouping

- **Grouping** refers to the order in which C++ applies certain operators
- In the expression `a + b + c` the `+` operators group from left-to-right as if we had written `(a + b) + c`
- Most C++ operators of the same precedence group left-to-right
- Note: Some operators group right-to-left

Decision Making: Equality and Relational Operators

- We now introduce C++'s `if` statement, which allows a program to perform different actions based on whether a **condition** is true or false.

Decision Making: Equality and Relational Operators (Cont.)

- Conditions in `if` statements can be formed by using the **relational operators** and **equality operators** in the following table:

Algebraic relational or equality operator	C++ relational or equality operator	Sample C++ condition	Meaning of C++ condition
<i>Relational operators</i>			
$>$	<code>></code>	<code>x > y</code>	x is greater than y
$<$	<code><</code>	<code>x < y</code>	x is less than y
$\geq$	<code>>=</code>	<code>x >= y</code>	x is greater than or equal to y
$\leq$	<code><=</code>	<code>x <= y</code>	x is less than or equal to y
<i>Equality operators</i>			
$=$	<code>==</code>	<code>x == y</code>	x is equal to y
$\neq$	<code>!=</code>	<code>x != y</code>	x is not equal to y

Decision Making: Equality and Relational Operators (Cont.)

- Reversing the order of the symbols in the operators $\neq$, $\geq$ and $\leq$ (by writing them as $=!$, $=>$ and $=<$, respectively) is usually a syntax error.
- In some cases, writing $\neq$ as $=!$ will not be a syntax error but almost certainly will be a **runtime logic error**—an error that causes the program to execute incorrectly.
 - This will be covered in the discussion of logical operators.

Decision Making: Equality and Relational Operators (Cont.)

- Confusing `==` and `!=`

- Confusing the equality operator `==` with the assignment operator `=` results in logic errors.
- We like to read the equality operator as “is equal to” or “double equals” and the assignment operator as “gets” or “gets the value of” or “is assigned the value of.”
- Confusing these operators may not necessarily cause an easy-to-recognize syntax error, but it may cause **subtle logic errors**. Compilers generally **warn** about this.

Decision Making: Equality and Relational Operators (Cont.)

- Using the `if` Statement

- The next program uses six statements to compare two integers input by the user.
- If a given `if` statement's condition is true, the output statement in the body of that statement executes.
- If the condition is false, the output statement in the body is skipped.

Decision Making: Equality and Relational Operators (Cont.)

```
1 // Comparing integers using if statements, relational operators
2 // and equality operators.
3 #include <iostream> // enables program to perform input and output
4
5 using std::cout; // program uses cout
6 using std::cin; // program uses cin
7
8 // function main begins program execution
9 int main() {
10     int number1{0}; // first integer to compare (initialized to 0)
11     int number2{0}; // second integer to compare (initialized to 0)
12
13     cout << "Enter two integers to compare: "; // prompt user for data
14     cin >> number1 >> number2; // read two integers from user
15
16     if (number1 == number2) {
17         cout << number1 << " == " << number2 << "\n";
18     }
19
20     if (number1 != number2) {
21         cout << number1 << " != " << number2 << "\n";
22     }
23
24     if (number1 < number2) {
25         cout << number1 << " < " << number2 << "\n";
26     }
27
28     if (number1 > number2) {
29         cout << number1 << " > " << number2 << "\n";
30     }
31
32     if (number1 <= number2) {
33         cout << number1 << " <= " << number2 << "\n";
34     }
35
36     if (number1 >= number2) {
37         cout << number1 << " >= " << number2 << "\n";
38     }
39 } // end function main
```

Enter two integers to compare: 3 7

3 != 7

3 < 7

3 <= 7

Enter two integers to compare: 22 12

22 != 12

22 > 12

22 >= 12

Enter two integers to compare: 7 7

7 == 7

7 <= 7

7 >= 7

Decision Making: Equality and Relational Operators (Cont.)

- `using` Declarations

- Lines 5–6

```
5 using std::cout; // program uses cout
6 using std::cin;  // program uses cin
```

are `using declarations` that eliminate the need to repeat the `std::` prefix as we did in earlier programs.

- We can now simply write `cout` instead of `std::cout` and `cin` instead of `std::cin` in the remainder of the program.

- `using` Directive

- In place of lines 5–6, many programmers prefer the directive: `using namespace std;` which enables your program to use names from the namespace without the qualification.

- **Note:** Using `std::` is considered good practice and can help you avoid some subtle coding errors.

Decision Making: Equality and Relational Operators (Cont.)

- Comparing Numbers

- The statement in lines 16–18

```
16  if (number1 == number2) {  
17      cout << number1 << " == " << number2 << "\n";  
18  }
```

determines whether the values of variables `number1` and `number2` are equal.

- ◆ If so, the output statement (line 17) displays a line of text indicating that the numbers are equal.
- For each condition that is true in the remaining statements starting in lines 20, 24, 28, 32 and 36, the corresponding output statement displays an appropriate line of text.

Decision Making: Equality and Relational Operators (Cont.)

- Common Logic Error: Placing a Semicolon after a Condition
 - Placing a semicolon immediately after the right parenthesis of the condition in an `if` statement is often a logic error (although not a syntax error).
 - The semicolon causes the body of the `if` statement to be empty, so the `if` statement performs no action, regardless of whether or not its condition is true.
 - Worse yet, the original body statement of the `if` statement now becomes a statement in sequence with the `if` statement and always executes, often causing the program to produce incorrect results. Compilers may issue a warning for this logic error.

Decision Making: Equality and Relational Operators (Cont.)

- Splitting Lengthy Statements
 - A lengthy statement may be spread over several lines. If you must do this, choose meaningful breaking points, such as after a comma in a comma-separated list or after an operator in a lengthy expression.
 - If a statement is split across two or more lines, it's a good practice to **indent** all subsequent lines.

Decision Making: Equality and Relational Operators.

- Operator Precedence and Grouping

- Except for the assignment operator =, all the operators presented in this chapter group from left to right.
- Assignments (=) group from right to left. So, an expression such as `x=y=0` evaluates as if it had been written `x=(y=0)`, which first assigns 0 to `y`, then assigns to `x` the result of that assignment—that is, the value of `y`, which is now 0.
- You can refer to the complete operator-precedence chart at https://en.cppreference.com/w/cpp/language/operator_precedence

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class

- Throughout this course, we emphasize using valuable preexisting classes from the C++ standard library and various libraries from the C++ open-source community.
 - You'll focus on knowing what libraries are out there, choosing the ones you'll need for your applications, creating objects from existing library classes and making those objects exercise their capabilities.
- By Objects Natural, we mean that you'll be able to conveniently create and program with powerful objects even before you learn to create your own custom classes.
 - You've already worked with C++ objects—specifically the `cout` and `cin` objects, which encapsulate the mechanisms for output and input, respectively.
 - These objects were created for you behind the scenes using classes from the header `<iostream>`. In this section, you'll create and interact with objects of the C++ standard library's class.

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

- Test-Driving Class `string`

- Classes cannot execute by themselves.

- ◆ A `Person` object can drive a `Car` object by telling it what to do (go faster, go slower, turn left, turn right, etc.) without knowing how the car's internal mechanisms work.
 - ◆ Similarly, the `main` function can “drive” a `string` object by calling its member functions—without knowing how the class is implemented. In this sense, `main` in the following program is referred to as a **driver program**.

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

```
1 // Standard library string class test program.
2 #include <iostream>
3 #include <string>
4 using namespace std;
5
6 int main() {
7     string s1("happy");
8     string s2(" birthday");
9     string s3; // creates an empty string
10
11     // display the strings and show their lengths (length is C++20)
12     cout << "s1: \"\" << s1 << "\"; length: \" << s1.length()
13         << "\ns2: \"\" << s2 << "\"; length: \" << s2.length()
14         << "\ns3: \"\" << s3 << "\"; length: \" << s3.length();
15
16     // compare strings with == and !=
17     cout << "\n\nThe results of comparing s2 and s1:" << boolalpha
18         << "\ns2 == s1: \" << (s2 == s1)
19         << "\ns2 != s1: \" << (s2 != s1);
20
21     // test string member function empty
22     cout << "\n\nTesting s3.empty():\n";
23
24     if (s3.empty()) {
25         cout << "s3 is empty; assigning to s3;\n";
26         s3 = s1 + s2; // assign s3 the result of concatenating s1 and s2
27         cout << "s3: \"\" << s3 << "\";
28     }
29
30     // testing new C++20 string member functions
31     cout << "\n\ns1 starts with \"ha\": \" << s1.starts_with("ha") << "\n";
32     cout << "s2 starts with \"ha\": \" << s2.starts_with("ha") << "\n";
33     cout << "s1 ends with \"ay\": \" << s1.ends_with("ay") << "\n";
34     cout << "s2 ends with \"ay\": \" << s2.ends_with("ay") << "\n";
35 }
```

s1: "happy"; length: 5
s2: " birthday"; length: 9
s3: ""; length: 0

The results of comparing s2 and s1:

s2 == s1: false

s2 != s1: true

Testing s3.empty():

s3 is empty; assigning to s3;

s3: "happy birthday"

s1 starts with "ha": true

s2 starts with "ha": false

s1 ends with "ay": false

s2 ends with "ay": true

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

● Instantiating Objects

- Typically, you cannot call a member function of a class until you create an object of that class—also called instantiating an object.
- Lines 7–9

```
7   string s1{"happy"};  
8   string s2{" birthday"};  
9   string s3; // creates an empty string
```

create three `string` objects:

- ◆ `s1` is initialized with a copy of the string literal `"happy"`,
- ◆ `s2` is initialized with a copy of the string literal `" birthday"`
- ◆ `s3` is initialized by default to the empty (that is, `""`).

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

- Instantiating Objects

- When we declare `int` variables, as we did earlier, the compiler knows what `int` is—it's a fundamental type built into C++.
- In lines 8–10, however, the compiler does not know in advance what type `string` is—it's a class type from the C++ standard library.
- When packaged properly, classes can be reused by other programmers.
 - ◆ This is one of the most significant benefits of working with languages like C++ that support object-oriented programming.
 - ◆ Such languages have rich libraries of powerful prebuilt classes.
 - ◆ For example, you can reuse the C++ standard library's classes in any program by including the appropriate headers—in this case, the `string` **header** (line 4). The name `string`, like the name `cout`, belongs to namespace `std`.

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

- `string` Member Function `length`

- Lines 12–14 output each `string` and its length.

```
12      cout << "s1: \" << s1 << "\"; length: \" << s1.length()  
13      << "\ns2: \" << s2 << "\"; length: \" << s2.length()  
14      << "\ns3: \" << s3 << "\"; length: \" << s3.length();
```

- The `string` class's `length` member function returns the number of characters stored in a particular `string` object.
- In line 12, the expression returns 's length by calling the object's `length` member function.
 - ◆ To call this member function for a specific object, you specify the object's name (`s1`), followed by the dot operator (`.`), then the member function name (`length`) and a set of parentheses.
 - ◆ Empty parentheses indicate that `length` does not require additional information to perform its task.
 - ◆ Note that some member functions require additional information called arguments to perform their tasks.

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

- `string` Member Function `length`

- From `s1`'s view, when the `length` member function is called:

1. The program transfers execution from the call (line 12) to member function `length`.
 - Because `length` was called via the `s1` object, `length` “knows” which object’s data to manipulate.
2. Next, member function `length` performs its task—that is, it returns `s1`’s length to line 12, where the function was called.
 - The `main` function does not know how `length` performs its task, just as the driver of a car doesn’t know how engines are implemented.
3. The `cout` object displays the number of characters returned by member function `length`, then the program continues executing, displaying the `string` `s2`, `s3` and their lengths.

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

- Comparing `string` Objects with the Equality Operators

- Like numbers, `strings` can be compared with one another.
- Lines 17–19 compare `s2` to `s1` using the equality operators—`string` comparisons are case sensitive.

```
17      cout << "\n\nThe results of comparing s2 and s1:" << boolalpha
18          << "\ns2 = s1: " << (s2 == s1)
19          << "\ns2 != s1: " << (s2 != s1);
```

- Usually, when you output a condition's value, C++ displays 0 for false or 1 for true. The stream manipulator `boolalpha` (line 17) from the `<iostream>` header tells the output stream to display condition values more naturally as the words `false` or `true`.

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

- `string` Member Function `empty`
 - Line 24 calls `string` member function `empty`, which returns `true` if the `string` is empty—that is, the length of the `string` is 0. Otherwise, `empty` returns `false`.
 - The object `s3` was initialized “by default” to the empty string, so it is indeed empty, and the body of the `if` statement will execute.

```
24  if (s3.empty()) {  
25      cout << "s3 is empty; assigning to s3;\n";  
26      s3 = s1 + s2; // assign s3 the result of concatenating s1 and s2  
27      cout << "s3: \"" << s3 << "\"";  
28  }
```

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class (Cont.)

- `string` Concatenation and Assignment
 - Line 26 assigns a new value to `s3` produced by “adding” the strings `s1` and `s2` using the `+` operator—known as `string` concatenation.
 - After the assignment, `s3` contains the characters of `s1` followed by the characters of `s2`— “happy birthday”.
 - Line 27 outputs `s3` to demonstrate that the assignment worked correctly.

```
24  if (s3.empty()) {  
25      cout << "s3 is empty; assigning to s3;\n";  
26      s3 = s1 + s2; // assign s3 the result of concatenating s1 and s2  
27      cout << "s3: \"" << s3 << "\"";  
28  }
```

Objects Natural Case Study: Creating and Using Objects of Standard-Library Class.

- C++20 `string` Member Functions
`starts_with` and `ends_with`

- Lines 30–34

```
30 // testing new C++20 string member functions
31 cout << "\n\ns1 starts with \"ha\": " << s1.starts_with("ha") << "\n";
32 cout << "s2 starts with \"ha\": " << s2.starts_with("ha") << "\n";
33 cout << "s1 ends with \"ay\": " << s1.ends_with("ay") << "\n";
34 cout << "s2 ends with \"ay\": " << s2.ends_with("ay") << "\n";
```

demonstrate C++20 `string` member functions `starts_with` and `ends_with`, which return `true` if the string starts with or ends with a specified substring, respectively; otherwise, they return `false`.

- ◆ Lines 31–32 show that `s1` starts with `"ha"` but `s2` doesn't.
- ◆ Lines 33–34 show that `s2` end with `"ay"` but `s1` doesn't.